FIT5149 S2 2026 Assessment 1

Sanctioned Loan Amount Prediction

Assessment Information	**Details**
Marks	25% of all marks for the unit
Due Date	11:55 PM, Monday 14 September 2026
Extension	Extensions and other individual alterations to the assessment regime will only be considered using the University Special Consideration Policy. Students should carefully read the Special Consideration website, especially the details about required formal documentation. All special consideration requests should be made using the Special Consideration Application. Note that students will no longer seek extensions from the teaching team/Chief Examiner. All extensions will be via the special consideration form process, and extensions will be approved by SEBS (not the Chief Examiner).
Lateness	For all assessment items handed in after the official due date without an agreed extension, a 5% penalty applies to the student's mark for each day after the due date (including weekends and public holidays) for up to 7 days. Assessment items handed in after 7 days will not be considered/marked.
	Continued on next page

Assessment Information (continued)

Group Member	This assessment is completed in groups of **two**. Your partner does not have to be enrolled in the same applied class as you. If you are unable to find a partner, you may submit individually. This is permitted but **not encouraged**: the workload is set for two people and is not reduced for individual submissions. Groups are created and managed by students themselves, through the group selection activity under Assignment 1 on Moodle. **Groups must be finalised by 11:55 PM, Saturday 5 September 2026.**
Authorship	The final submission must be **identifiably your own work**. Breaches of this requirement will result in an assignment not being accepted for assessment and may result in disciplinary actions.
Submission	Submission is via Moodle. See Section 6 for the complete list of required files. A draft submission will not be marked.
Programming language	Python or R

Note: Please read this description carefully from start to finish before you begin your work.

1 Introduction

When a consumer lender receives a loan application, the amount the applicant asks for and the amount the lender is ultimately willing to commit are rarely the same. The lender must weigh the applicant's capacity to service the debt, the security offered, the applicant's credit history, and its own risk appetite, and then arrive at a figure. Automating part of this judgement is one of the most common applications of predictive modelling in retail finance: it speeds up decisions for applicants, makes the process more consistent between assessors, and lets human underwriters concentrate on the cases that genuinely need judgement.

This assessment places you in the position of a data scientist at Yarra Credit Group, an Australian consumer lender. You are given a historical extract of processed loan applications and asked to build a model that predicts the amount sanctioned on a new application.

The data is real operational data in the sense that matters for this unit: it was assembled from several internal systems, it carries the inconsistencies that come with that, and nobody cleaned it up for you before it landed on your desk. Part of the work of this assessment—and a substantial part of the marks—is establishing what you are actually looking at before you model it.

A note on how this assessment is weighted. The largest share of the marks is for your report, not for your leaderboard position. A well-argued analysis built on a modest model will score considerably better than a strong model with no explanation of how you arrived at it. Section 4 sets out the breakdown in full.

2 Task Description

2.1 Prediction Task

Build a regression model that predicts `sanctioned_amount_aud`, the amount in Australian dollars sanctioned on each loan application, using the features supplied in the dataset.

General requirements.

- Develop and compare at least **three different types** of model. Different feature sets or hyperparameter settings for the same algorithm do not count as different types—linear regression with two different feature sets is one type; linear regression and a gradient boosting regressor are two.
- Apply appropriate data preprocessing, feature engineering and feature selection.
- Design and justify a validation strategy, and use it—not the public leaderboard—to select your final model. See Section 3 for why this distinction matters.
- Select one final model, retrain it on all available labelled data, and generate predictions for the test set.

2.2 Analysis Task

- Establish what condition the data is in before you model it. Document what you find, what you decided to do about it, and what effect your decisions had on your results. Assertions about the data should be supported by evidence you generated yourself.
- Identify the input variables that most influence the sanctioned amount, and provide statistical evidence for your conclusions rather than relying on a single feature-importance plot.
- Explain which features—including any you constructed—are useful and why, in terms of the lending problem rather than only in terms of model output.
- Discuss the limitations of your final model. Where does it perform poorly, and what would you want before deploying it?

2.3 Evaluation Metric

Given a test set $\{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$ and predictions $\{\hat{y}_1, \ldots, \hat{y}_n\}$, performance is measured by the **root mean squared error**:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

RMSE is expressed in Australian dollars and can be read as the typical size of the error your model makes on an application. Lower is better.

In Python, RMSE is available via `sklearn.metrics.mean_squared_error` followed by a square root; in R, via `Metrics::rmse`.

Because RMSE squares each error before averaging, a small number of large mistakes affects it far more than a large number of small ones. Your report should discuss whether this is the right property for this problem, and what would change if a different metric were adopted.

3 Kaggle Competition

A Kaggle competition is run for this assessment, at:

https://www.kaggle.com/competitions/fit5149-s2-2026-assessment-1

The link is also available in the Assessment section of Moodle. **The competition closes at the assessment deadline given on page 1.**

If you are granted special consideration, or submit late under the lateness policy, you will not be able to enter the competition after it has closed. In that case your model will be

re-run by the teaching team and scored on the same private test set, and the marks in Sections 3.5 and 3.6 will be awarded against the same thresholds and the same ranking as everyone else. The requirements in Section 3.7 apply unchanged: you must still nominate which model is your final one, and it must be reproducible from your submitted code. Your leaderboard result is worth **30 marks out of 100**, awarded in two parts:

- **15 marks** for the performance level you reach, measured against published baselines (Section 3.5).
- **15 marks** for your position in the final ranking (Section 3.6).

3.1 How the leaderboard works

You will submit predictions for all 10,000 rows of the test set in a single file. Kaggle scores your submission twice:

- The **public leaderboard** score, visible to you immediately, is computed on 2,000 of those rows.
- The **private leaderboard** score, hidden until the competition closes, is computed on the remaining 8,000 rows. **Your marks are based on the private score only.**

You will not be told which rows fall into which group.

3.2 Accounts and identification

Form your Kaggle team by the same date as your Moodle group. Kaggle closes team formation before the competition ends, and a team that has not been formed by then cannot be merged afterwards.

Each group must compete under a **single Kaggle account or team**. Entering from more than one account—whether to increase the effective number of daily submissions, to probe the leaderboard, or for any other reason—is a breach of the competition rules and will be treated as academic misconduct.

Your report must state your Kaggle team name and the username of every group member. Marks are awarded against the identity given in the report. If it is missing, or does not correspond to an entry on the private leaderboard, **all 30 marks under this section will be awarded as zero.** Making sure the two match is your responsibility, and no marks will be awarded on the basis of an entry we cannot verify.

3.3 Submission limits

You may make at most **3 submissions per day**. There is no limit on the total number of submissions over the assessment period.

3.4 The public leaderboard is not a validation set

Read this subsection carefully; it describes a mistake that is easy to make and costly to correct.

The public leaderboard is computed on approximately 2,000 rows. At that sample size the standard deviation of RMSE due to sampling alone is roughly $1,700, which is comparable to—or larger than—the genuine difference in quality between a competent model and an excellent one. Movement in your public score between submissions is therefore substantially noise, and a model selected because it scored well on the public leaderboard is partly selected on that noise.

By contrast, you have 19,322 labelled training rows. A properly constructed cross-validation on that data gives you a far more reliable estimate of model quality than the public leaderboard ever can. **Use your own validation, not the leaderboard, to choose your final model.**

Marks are attached to this. Section 4 allocates marks to your model selection and validation strategy. Selecting your final model primarily on the basis of leaderboard feedback, rather than on a validation scheme you designed and can defend, will be penalised in that component regardless of the score you achieve.

3.5 Performance against baselines (15 marks)

These 15 marks are awarded in bands, not by rank. Every threshold is fixed in advance and is the same for everyone. The teaching team has developed a baseline model for this problem, and the thresholds are set with reference to it. RMSE is measured in Australian dollars, and **lower is better**.

Private RMSE (AUD)	Marks
above 56,700	0
48,400 to 56,700	6
38,000 to 48,400	10
33,000 to 38,000	13
at or below 33,000	15

3.6 Ranking by private RMSE (15 marks)

The remaining 15 marks are awarded according to your position on the final private leaderboard. All entries are ranked by private RMSE and divided into bands by percentile.

These marks are separate from those in Section 3.5. A group's performance, as evaluated against the baselines in Section 3.5, is not influenced by its ranking under this section.

Position on the private leaderboard	Marks
Top 10%	15
Next 20% (10–30%)	12
Next 30% (30–60%)	10
Next 20% (60–80%)	6
Bottom 20%, where Section 3.5 awarded 13 or more	3
Bottom 20%, otherwise	0

The following rules apply to the ranking:

- Each group occupies a single position, regardless of whether it has one or two members. Groups of one and groups of two are ranked in the same pool.
- Groups that make no valid submission before the deadline receive 0 for this component and are excluded from the ranking used to determine the percentile bands.
- Groups are ranked by private RMSE. Where two or more groups record the same private RMSE, they share the same rank position, and each receives the marks of the most favourable band into which any of them falls.

3.7 Final submissions and verification

Kaggle allows you to nominate up to **two** of your submissions as your final entries; these are the entries scored on the private leaderboard. Choose them before the competition closes. If you do not make a selection, Kaggle applies its own default, which may not be the entry you intended.

The model you submit on Moodle must be the one that produced one of those two nominated entries. You may not develop a third model and submit that instead.

After the deadline, submitted code will be re-run and the resulting predictions compared against your nominated Kaggle entries. Your code is treated as reproducing your entry if the re-run predictions match one of them to within **5% of its private RMSE**. This tolerance is deliberately wide: small differences arising from library versions or non-deterministic training are expected and will not be penalised.

If your submitted code does not reproduce either of your nominated entries within that tolerance, all 30 marks under this section will be awarded as zero. To avoid this:

- Fix a random seed everywhere randomness enters your pipeline.
- Confirm your notebook runs top to bottom from a fresh kernel, on the supplied data files, without manual intervention.
- Record the package versions you used in your `README`.
- Check that the model you package is the one that generated your nominated entry, not a later or earlier variant.

4 Marking

Component	Marks
Data quality investigation and exploratory analysis	20
Preprocessing and feature engineering	15
Modelling, model selection and validation strategy	20
Results analysis, interpretation and critique	15
Kaggle competition (Section 3)	30
Total	**100**

A detailed rubric will be released separately on Moodle.

Two components deserve specific comment, because they are where reports most often lose marks.

Data quality investigation. Marks here are for what you found and how you established it, not for reciting a checklist. A statement such as "we checked for missing values" earns nothing on its own. A statement such as "`column_x` is missing for 24% of rows, and the missingness rate varies by a factor of four across categories of `column_y`, which rules out a simple mean imputation because ..." earns full marks, provided your submitted code reproduces the numbers.

Model selection and validation strategy. Your report must state how the final model was chosen. This includes the design of your validation scheme (how many folds, whether stratified, how preprocessing was fitted so as to avoid leakage between folds), what candidates you compared, on what basis, and how the validation scores you report correspond to the model you actually submitted. Every number you quote must be reproducible from your submitted code.

5 Submission Guidelines and Hints

Based on submissions in previous semesters:

- Use relative paths in your notebook (e.g. `./data/training_set.csv`). Place data files in a folder relative to your notebook rather than referring to an absolute path on your own machine.
- Fix a random seed so that your results are reproducible.
- Before submitting, restart the kernel and run the whole notebook top to bottom. If it does not run cleanly from a fresh kernel, marks under reproducibility will be lost.
- Provide interpretation and discussion of your results, especially for plots and statistics. A figure with no accompanying reasoning earns very little.

- Choose visualisations that convey information relevant to your argument, rather than producing every plot your library can generate.
- Avoid excessively long notebooks. Focus on the analyses that support your conclusions and remove exploratory dead ends.
- Use markdown cells to explain your reasoning as you go.
- Use the discussion forum and consultation sessions to clarify any doubts.

6 Submission

You are required to submit the following.

- **A PDF report**, named `groupID_ass1_report.pdf`. **Maximum 5 pages**; if the report exceeds the limit, only the first 5 pages will be assessed. The report must document:
  - What you found when you investigated the data, and what you did about it.
  - Your preprocessing and feature engineering, with justification.
  - The models you compared, your validation design, and how you selected the final model.
  - Your results, including performance on your own validation scheme and on the leaderboard, with discussion of any discrepancy between them.
  - Analysis of which features matter and why, with statistical evidence.
  - Limitations of your final model.
  - **Your Kaggle team name and the username of every group member.** Your group must use a single Kaggle account or team, and marking is carried out against the identity stated here. **If this is missing or does not match an entry on the private leaderboard, all 30 marks under Section 3 will be zero.** See Section 3.2.

  Material that does not fit within the 5 pages—a reference list, supporting tables, additional figures—may be placed in an appendix after the page limit. **Appendices and the reference list are not assessed**, so any point you want marked must appear within the 5 pages.

- **A project plan and statement of work division**, placed at the end of the PDF, completed using the template provided in the Assessment section of Moodle. This section sits **outside** the 5-page limit.

  The teaching team reserves the right to adjust individual marks to reflect actual contribution. Where the statement of work division, the submitted materials, or a discussion with group members indicates that the workload was not shared as claimed, members of the same group may receive different marks.

- **A ZIP file** named `groupID_ass1_impl.zip` containing:

- The implementation of your **final** model. Code for models you considered but rejected should be removed; the comparison belongs in the report. Keep a copy of that code yourself in case an interview is scheduled.
- `submission.csv`, the prediction file you submitted to Kaggle. It must be reproducible by the assessor by running your submitted code.
- A `README` explaining how to set up and run your code, including package versions.
- The signed assignment cover sheet.
- The Generative AI declaration form.
- The AI chat history file, if applicable.

Generative AI Usage Guideline

You may use generative AI for grammar checking or paraphrasing, and for learning concepts. Using generative AI to produce code or results directly for this assessment is prohibited. If you use AI, include the chat history with your submission. Whether or not you use AI, you must complete the declaration file.

7 Academic Integrity

Please be aware of the University's policy on academic integrity. Monash University takes academic misconduct[1] very seriously. You may learn from published material and understand the principles behind an analysis, but you must complete this assessment with your own work.

The dataset used in this assessment has been constructed specifically for this unit. Submitting predictions obtained from any source other than a model you built yourself, or attempting to obtain the test labels by any means other than modelling, will be treated as academic misconduct.

A Appendix: Dataset

A.1 Files provided

```
data/
├── training_set.csv # labelled applications, including the target
├── kaggle_test_X.csv # unlabelled applications for prediction
└── sample_submission.csv # required submission format
```

[1] https://www.monash.edu/students/study-support/academic-integrity

File	Rows	Columns
training_set.csv	19,322	29
kaggle_test_X.csv	10,000	28

kaggle_test_X.csv contains the same columns as training_set.csv with the target column removed.

A.2 Submission format

submission.csv must contain exactly two columns and one row per application in kaggle_test_X.csv:

application_id	sanctioned_amount_aud
LN-000123	54812.90
LN-000456	21044.35
...	...

Use sample_submission.csv as your template. Kaggle will reject a file with the wrong number of rows, missing identifiers, or non-numeric predictions.

A.3 Data dictionary

Column	Type	Description
application_id	identifier	Unique reference for each application, in the format LN-######. Not a predictor.
applicant_gender	categorical	Gender recorded on the application. Values: female, male.
applicant_age	numeric	Age of the applicant in years at the time of application.
annual_income_aud	numeric	Annual income declared by the applicant, in AUD.
monthly_income_aud	numeric	Monthly income figure recorded by the assessing officer, in AUD.
income_consistency	categorical	Assessor's judgement of how consistent the applicant's income is. Values: steady, variable.
occupation_class	categorical	Broad employment category of the applicant.
employment_sector	categorical	Industry sector of the applicant's employment. 18 categories.

Data dictionary (continued)

Column	Type	Description
residence_area	categorical	Classification of the applicant's residential area. Values: inner_metro, outer_metro, regional.
requested_amount_aud	numeric	Loan amount requested by the applicant, in AUD.
existing_repayments_aud	numeric	Applicant's current periodic repayment commitments on existing credit, in AUD.
expense_flag_a	categorical	Internal expense indicator recorded during assessment. Values: yes, no.
expense_flag_b	categorical	A second internal expense indicator. Values: yes, no.
dependants_count	numeric	Number of dependants declared by the applicant.
credit_rating	numeric	Credit assessment score supplied by the credit bureau. Higher values indicate lower assessed risk.
prior_default_count	numeric	Number of prior defaults recorded against the applicant.
card_status	categorical	Status of the applicant's credit card holdings. Values: current, dormant, none_held, unknown.
enquiry_count_12m	numeric	Number of credit enquiries recorded against the applicant in the preceding twelve months.
risk_review_flag	numeric	Indicator set when the application was referred for additional risk review. Values: 0, 1.
property_ref	identifier	Internal reference number for the security property.
property_age_years	numeric	Age of the security property, in years.
property_category	categorical	Internal classification of the security property. Values: cat_a to cat_d.
property_area	categorical	Classification of the area in which the security property is located. Values: inner_metro, outer_metro, regional.
property_value_aud	numeric	Assessed value of the security property, in AUD.
has_co_applicant	numeric	Indicator for whether the application includes a co-applicant.
branch_id	categorical	Branch at which the application was lodged. Approximately 200 branches.

Data dictionary (continued)

Column	Type	Description
`marketing_channel`	categorical	Channel through which the applicant was acquired.
`application_date`	date	Date the application was lodged. Format: `DD/MM/YYYY`.
`sanctioned_amount_aud`	**target**	Amount sanctioned on the application, in AUD.

B Appendix: Mini Tutorial on Supervised Learning in Practice

B.1 Overview

Supervised learning in practice typically consists of three major steps: preparing and representing the data, selecting a model, and applying the selected model to predict values for unseen data. The dataset supplied here provides raw features; you are expected to apply preprocessing and feature engineering to turn them into representations suited to the models you choose. Model selection involves identifying appropriate model families and tuning their hyperparameters. Both stages matter, and in practice they interact: a change in how the features are represented often changes which model performs best.

B.2 Recommended Practice

The following steps are recommended not only for completing this assessment but as a general approach to supervised learning problems on tabular data.

(i) **Look at the data before you model it.** Compute summary statistics for every column. Plot the distribution of the target and of each numeric feature. Cross-tabulate categorical features against each other and against missingness indicators. Check for duplicated rows and for columns that are redundant with one another. This step routinely determines the ceiling on everything that follows.

(ii) **Set aside a validation scheme early.** Split the labelled data, or set up cross-validation, before you begin tuning. Fit every preprocessing step—imputation, scaling, encoding—inside the cross-validation loop rather than on the full dataset, or your validation scores will be optimistic. In `scikit-learn`, `Pipeline` combined with `cross_val_score` handles this correctly; fitting an imputer on the full training set before splitting does not.

(iii) **Represent each variable appropriately.** This dataset contains binary, nominal, ordinal, numeric and date variables.

- **Nominal variables.** `pandas.get_dummies()` or `sklearn.preprocessing.OneHotEncoder`. For a variable with many categories, one-hot encoding may not be practical; alternatives include grouping rare categories, ordinal encoding for tree-based models, or target encoding. If you use target encoding, compute it out-of-fold—computing it on the full training set leaks the target into your features and will inflate your validation score without improving your leaderboard score.
- **Date variables.** Parse the string into a datetime first, taking care with the day/month order. From there you can extract ordinal features (month, quarter, year), cyclical features such as $\sin(2\pi \cdot \text{month}/12)$ and $\cos(2\pi \cdot \text{month}/12)$, or elapsed time since a reference date.
- **Numeric variables.** Consider standardisation ($\frac{x-\mu_x}{\sigma_x}$, via `StandardScaler`), rescaling to a fixed range (`MinMaxScaler`), log transformation for skewed variables (`np.log1p`), or binning (`pandas.cut`). Which of these help depends on the model: linear models are sensitive to scale, tree-based models are not.
- **Missing values.** Decide on a strategy per column rather than applying one rule everywhere, and justify the decision. Options include listwise deletion, simple imputation with a summary statistic, model-based or iterative imputation (`sklearn.impute.IterativeImputer`), adding an explicit missingness indicator, or using a model that handles missing values natively (`HistGradientBoostingRegressor` does). These choices are not equivalent, and which is appropriate depends on why the values are missing.

(iv) **Tune hyperparameters against your validation scheme.** Common approaches:

- *Manual search.* Choose values based on judgement and previous results. Useful for developing intuition about which ranges are worth exploring.
- *Grid search.* Evaluate all combinations from a specified set of values. Thorough but expensive.
- *Random search.* Sample a fixed number of combinations at random. Usually achieves comparable results to grid search at a fraction of the cost when the budget is limited.

See https://scikit-learn.org/stable/modules/grid_search.html for details. In practice these are combined: manual exploration to find promising ranges, then random or grid search within them.

(v) **Retrain the selected model on all labelled data** before generating your final predictions, and confirm your submission file has the correct format before uploading.

Steps (i), (iii) and (iv) are applied iteratively: a change in feature representation may change which model performs best, and a substantial change in preprocessing usually means redoing part of the model selection.